Course: MA3206 AI

Assignment 5

Deadline: 11:59 pm, 02 April, 2026

State & Action Space

- States S = { High, Low, Charging } — the robot's current battery level
- Actions A = { Search, Wait } — available in every state
- Discount factor $\gamma = 0.9$
- Convergence threshold $\theta = 1 \times 10^{-6}$

Transition Probabilities P(s' | s, a)

State s	Action a	Next State s'	P(s'|s,a)	Reward R(s,a,s')
High	Search	High	0.7	+4
High	Search	Low	0.3	+4
High	Wait	High	1.0	+1
Low	Search	High	0.4	−3 (recharge penalty)
Low	Search	Low	0.6	+4
Low	Wait	Low	1.0	+1
Charging	Wait	High	1.0	0

Represent the MDP as follows in your code

- STATES = ['High', 'Low', 'Charging'] → index 0, 1, 2
- ACTIONS = ['Search', 'Wait'] → index 0, 1
- P — a 3D NumPy array of shape (3, 2, 3) where P[s, a, s'] = transition probability
- R — a 2D NumPy array of shape (3, 2) where R[s, a] = immediate reward
- Build P and R yourself by reading the table above.

Task 1 | Iterative Policy Evaluation

Concept

Given a fixed policy π, Policy Evaluation computes $V\pi(s)$ — the expected cumulative discounted reward starting from state s and always following π. We find $V\pi$ by repeatedly applying the Bellman Expectation update until values stop changing.

$$V(s) \leftarrow \Sigma_{s'} P(s' \mid s, \pi(s)) \times [\, R(s, \pi(s)) + \gamma \cdot V(s') \,]$$

What to Implement

Task 1.1 — Build the MDP

Define the MDP in Python. Create the STATES list, ACTIONS list, P array of shape (3, 2, 3), and R array of shape (3, 2) using the transition table above. Set GAMMA = 0.9.

Verify your setup by printing:

- The sum of P[s, a, :] for every (s, a) pair — each must equal 1.0
- The full R array

Task 1.2 — Implement policy_evaluation()

Write a function with this signature:

def policy_evaluation(P, R, policy, gamma, theta)

The function must:

- Initialise V(s) = 0 for all states
- Repeatedly sweep through all states and apply the Bellman update
- Stop when max_s |V_new(s) − V_old(s)| < θ across all states
- Print the number of iterations taken
- Return the converged value array V

Evaluate the following policy π:

- High → Search, Low → Wait, Charging → Wait
- Print $V\pi(s)$ for all three states

Task 1.3 — Plot

Create a bar chart of $V\pi(s)$ for all three states.

Bonus :
Implement the same MDP using Gymnasium and compare the learned policy with the analytical solution.

Task 2 | Value Iteration

Concept

Value Iteration finds the optimal value function V* directly — without fixing a policy first. At each sweep it applies the Bellman Optimality update, taking the maximum over all actions. Once V* converges, the optimal policy π* is extracted with a single greedy step.

$$V(s) \leftarrow \max_a \Sigma_{s'} P(s'|s,a) \times [R(s,a,s') + \gamma \cdot V(s')]$$

What to Implement

Task 2.1 — Implement value_iteration()

Write a function with this signature:

def value_iteration(P, R, gamma, theta)

The function must:

- Initialise V(s) = 0 for all states
- At each sweep, compute Q(s, a) for every (state, action) pair
- Set V(s) = max over actions of Q(s, a)
- Stop when max change across all states < θ
- Store V after every iteration in a history list (needed for plotting)
- Return the converged V* and the history list

Task 2.2 — Extract the Optimal Policy

Write a function with this signature:

def extract_policy(V_star, P, R, gamma)

For each state, compute Q(s, a) for all actions using V* and return the action with the highest Q value. Print the optimal policy π*(s) for all three states.

Task 2.3 — Plot Convergence

Using the history list returned by value_iteration(), plot V(s) across iterations for all three states on the same graph (one line per state).

Task 3 | Policy Iteration

Concept

Policy Iteration alternates between two steps until the policy stabilises:

- Policy Evaluation — compute V$^{\pi}$b for the current policy
- Policy Improvement — update the policy greedily using the computed V$^{\pi}$b

Because there are finitely many deterministic policies and each improvement step is guaranteed not to make things worse, Policy Iteration always converges to π* in a small number of iterations.

What to Implement

Task 3.1 — Implement policy_improvement()

Write a function with this signature:

def policy_improvement(V, P, R, gamma)

For each state, compute Q(s, a) for all actions using the provided V, then return the greedy action. Also return a boolean stable that is True if the new policy is identical to the old one.

Task 3.2 — Implement the Full policy_iteration() Loop

Write a function with this signature:

def policy_iteration(P, R, gamma, theta)

The loop must:

- Start from the initial policy: all states take Wait
- Call policy_evaluation() to get V$^{\pi}$b for the current policy
- Call policy_improvement() to get a new policy
- Stop when stable = True (policy did not change)
- At each iteration, print the current policy and V values
- Store the policy and V at every iteration for plotting
- Return the final optimal policy, V*, and the histories

Task 3.3 — Plot Results

Produce two plots :

(a) Line plot — Vπ(s) for all three states across policy iteration steps (x-axis = iteration number, one line per state)

(b) Heatmap or table — the policy at each iteration (rows = states, columns = iterations, cell = action taken)

Note: Action 'Search' is NOT allowed in the Charging state.

Task 4 | Analysis and Interpretation

4.1 Compare Value Iteration and Policy Iteration:
- Which algorithm converges faster in this problem?
- Explain the difference in number of iterations.

4.2 Convergence Behavior:
- Plot and explain how V(s) changes over time.
- Why does Value Iteration update differ from Policy Evaluation?

4.3 Optimal Policy Interpretation:
- What is the optimal action in each state?
- Provide intuition behind why that action is optimal.

4.4 Practical Insight:
- In real-world systems (like battery robots), why is such a policy useful?

Submission Guidelines

- Submit well-commented Python code
- Include all plots with proper labels
- Provide brief explanations (2-3 lines) for each result
- Ensure code is clean and modular